IR-Programmable E-Ink Price Tags

Moeen Farhadi
Department of Electrical and Computer Engineering
University of Tehran
Tehran, Iran
moeenfarhadi@ut.ac.ir
810800019

***Abstract*—In recent years, there has been a huge growth in the need for display devices that use very little power while still being flexible and quick to update. One important use of this technology is Electronic Shelf Labels (ESLs), which are used in stores, warehouses, and industrial environments to show prices, stock levels, or management messages. E-Ink displays are a perfect choice for these systems because they use almost zero power when the image is static, have wide viewing angles, and look a lot like real paper. In this project, the goal is to design and build a programmable electronic shelf label that can be controlled through Infrared (IR) communication. To do this, we use an embedded system as the main control unit, an IR receiver to get commands, an E-Ink display to show the content. Combining these parts lets us build a simple, low-power, portable, and reliable system that can be used in a wide range of retail and industrial settings.**

***Index Terms*—Electronic Shelf Label (ESL)E-Ink Display,Embedded Systems,Infrared (IR) Communication,Low-Power Design.**

I. Introduction

A. Motivation and Overview

In modern retail and industrial logistics, keeping information like pricing and inventory up to date in real time is a constant challenge. Traditional paper labels are inefficient, labor-intensive, and wasteful. Electronic Shelf Labels (ESLs) solve this problem by allowing digital updates across thousands of items instantly. The main goal of this project is to implement a highly efficient, low-power, and programmable ESL system that can receive remote updates and retain its display state indefinitely, even during power interruptions.

B. Problem Definition and Constraints

To build a practical ESL system, the design must meet strict operational and hardware constraints. It needs to be portable, consume virtually zero power when idle, and maintain data persistence.

To achieve this, the system is constrained to the following architecture:

- **Central Control Unit**: A processor and system memory (RAM and ROM).
- **Display Technology**: An E-Ink Display to ensure high readability and zero static power consumption.
- **Peripherals and Connectivity**: An IR Receiver for wireless data configuration.
- **Power Management**(Not in the scope of this project): A Battery and Charging/Regulation Circuit to manage the portable power source efficiently.

This hardware framework ensures that the device remains simple, cost-effective, and capable of operating for extended periods on a single charge.

II. Embedded System Design and Components

The overall design in partitioned in 3 parts, the system, the peripherals and the firmware.

A. System

In this design, the system consists of a 32-bit RISC-V processor, one memory module, an SPI interface and a UART interface, all connected using a custom 8-bit bus.

1) Bus: The bus has a single master (the CPU), which reads from and writes to different addresses. To handle this transactions and handshakings, the following signals are used:

- Chip Select (CS) : Each module has a CS that is issued when it is the target of CPU. The CS is issued based on the address. The address range is explained in Table I.
- Read and Write : These signals indicate whether the type transaction is a read or a write.
- Ready : When a transaction begins, the slave pulls this line down until it gets the output ready or process the data in case of a write operation. During which the CPU waits until it goes back to 1.

TABLE I
Memory-mapped address ranges

Module	From	To	Size
Memory	0x00000000	0x0000FFFF	64KB
SPI	0x00010000	0x0001000F	16B
UART	0x00020000	0x000201FF	512B

2) CPU: As mentioned, the processor used in this design is a 32-bit RISC-V processor, specifically AFTAB [8] which is an implementation of the RV32IMZicsr standard. It follows the von neumann architecture for data and program memory. Interrupts are also supported and have been used in this design. Using dedicated read and write units, this 32-bit processor is able to talk over the 8-bit bus.

3) Memory: The memory used is an 8-bit wide 64KB memory with combinational reads and sequential writes. 64KB is chosen to accommodate for the firmware (Section II-C) and the processing data needed. This 64KB space is partitioned to multiple parts as shown in Table II

TABLE II
MEMORY REGIONS

Module	From	To	Size
Instructions	`0x00000000`	`0x00003FFF`	16KB
Data	`0x00004000`	`0x00007FFF`	16KB
Stack	`0x00008000`	`0x00009FFF`	8KB

4) UART: The design uses an opensource IP for the UART implementation wrapped with a custom logic that adds timeout timer and internal buffer. The UART IP is acquired from [5] and since it is only used to receive data from the IR sensor, only the `uart_rx.v` is used in the design. A wrapper (`uart_wrapper.v`) is made to add the following functionalities:

- **Timeout Timer** : A simple timer with interrupt to detect end-of-packet and interrupt the processor.
- **Buffer** : A 256-byte buffer made to store the incoming packet.
- **Bus Compatibility** : Signals necessary for communicating on the memory mapped bus are added.

5) SPI: To communicate with the SPI e-ink display, an SPI interface is needed, for this purpose an opensource IP from [6] is used. Again, just like the UART module, a wrapper (`SPI_Wrapper.v`) is used to make the SPI interface compatible with the memory mapped bus.

B. Peripherals

As the scope of this project is just simulation, to implement the e-ink display and the IR receiver, system-verilog emulators are written.

1) Display: To be able to observe the simulation and verify the functionality of the design, it is necessary to emulate the display such that the ESL's output is visible. For this purpose, a display emulator module (`disp.sv`) alongside a python script (`disp.py`) is used. The display module is connected to the output of the SPI interface in the testbench, upon receiving a command or data from the SPI, it passes them to the python script through a Linux IPC FIFO. The python script then executes the commands (such as putting a pixel or clearing the screen) and shows the result on a pygame window in real-time.

2) IR Receiver: The IR Receiver is used for UART-over-IR to send new tag data to the ESL. In order to emulate this component,the IR Reciever emulator (`IR.sv`) contains two strings which are the packets that are going to be sent to the design through UART. It uses the `uart_tx.v` module from [5] to transmit the characters of the two strings back to back. However there is a delay between sending the first and the second string in order to demonstrate the tag update funtionality.

C. Firmware

1) Main Loop: The firmware is responsible for capturing the tag data, processing it and updating the tag. Thus, it should be able to read UART data, process the incoming packet, generate new graphics and QR code, write them to the display via SPI. To do so, the code shown in Figure 1 from (`einkfirm.c`) is run in an infinite loop.

Here is the algorithm the program (Figure 1) follows:

1) Read UART for new data, it is done blocking.
2) Parse the packet into name, description, price, old price and website url
3) Clear the screen.
4) Using a QR code generator library [7] generate a QR code for the url.
5) Calculate the size of the QR code and find the X and Y coordinates that put it in the center of its box.
6) Render the QR code on screen at the calculated X and Y. Send the black pixels to the display using SPI.
7) Render the name of the product at the top left of the screen.
8) Render the description of the product right under the name.
9) Draw the lines dividing the 3 boxes.
10) Render the price of the product in the center of the price box.
11) If discounted, render the original price under the discounted price with a small font and a line crossing it.
12) Repeat.

2) Text Rendering: The text rendering is done using the `draw_text()` fuction. The bitmap font used to render text is generated by ChatGPT [4] and is stored in `font.h`. The characters are 5x7 pixels but they can be scaled using the an argument passed to `draw_text()`. This way we can achieve different font sizes.

`draw_text()` uses `draw_char()` to draw characters next to each other. `draw_char()` simply iterates over the 7x5 box at the coordinate it is provided and sets a black pixel where needed using `set_pixel`. `set_pixel` as shown in Figure 2, sends 6 bytes to the display over SPI. The first byte `0x00` is the command for writing a pixel. the next two 2-byte values are for the X and Y coordinates of the pixel. Finally the last byte is the color of the pixel (which is either 0 or 255 in this case).

`clear_screen()` is used to clear the screen, it simply sends the command byte `0x01` to clear the screen.

PorjectC - einkfirm.c

```c
size_t new_data_len = read_uart(new_data_buffer);
char *new_data_ptr = strtok(new_data_buffer, "|");
uint8_t buff_select = 0;
while (new_data_ptr != NULL)
{
  switch (buff_select)
  {
  case 0:
    strcpy(name_buffer, new_data_ptr);
    break;
  case 1:
    strcpy(desc_buffer, new_data_ptr);
    break;
  case 2:
    strcpy(price_buffer, new_data_ptr);
    break;
  case 3:
    strcpy(old_price_buffer, new_data_ptr);
    break;
  case 4:
    strcpy(url_buffer, new_data_ptr);
    break;

  default:
    break;
  }
  buff_select++;
  new_data_ptr = strtok(NULL, "|");
}


clear_screen();

size_t payloadLength = 0;
payloadLength += QrTinyWriteAlphanumeric(qr_buffer, payloadLength, url_buffer);
uint16_t formatInfo = QRTINY_FORMATINFO_MASK_000_ECC_LOW;
QrTinyGenerate(qr_buffer, payloadLength, formatInfo);

uint32_t qr_size = QRTINY_DIMENSION;
uint8_t qr_scale = 4;
for (uint8_t scale = 4; scale >= 0; scale--)
{
  if (qr_size * scale <= 104)
  {
    qr_scale = scale;
    break;
  }
}
uint32_t qr_x = (104 - qr_size * qr_scale) / 2;
uint32_t qr_y = 152 - qr_size * qr_scale - (104 - qr_size * qr_scale) / 2;

for (int y = 0; y < QRTINY_DIMENSION; y++)
{
  for (int x = 0; x < QRTINY_DIMENSION; x++)
  {
    int module = QrTinyModuleGet(qr_buffer, formatInfo, x, y);
    if (module)
    {
      for (uint32_t dy = 0; dy < qr_scale; dy++)
        for (uint32_t dx = 0; dx < qr_scale; dx++)
          set_pixel(qr_x + x * qr_scale + dx, qr_y + y * qr_scale + dy, module);
    }
  }
}
// Make and Render QR Code
draw_text(NAME_X, NAME_Y, name_buffer, NAME_FONT_SIZE);

uint32_t desc_y = 5 + 7 * NAME_FONT_SIZE + NAME_Y;
draw_text(NAME_X, desc_y, desc_buffer, DESC_FONT_SIZE);

uint32_t name_line_y = 5 + desc_y + DESC_FONT_SIZE * 7;
for (uint32_t x = 0; x < DISP_WIDTH; x++)
  set_pixel(x, name_line_y, 1);
for (uint32_t y = name_line_y; y < 152; y++)
  set_pixel(104, y, 1);

uint32_t price_width = strlen(price_buffer) * 6 * PRICE_FONT_SIZE - PRICE_FONT_SIZE;
uint32_t price_width_half = price_width / 2;
uint32_t price_offset = 192 / 2 - price_width_half;
uint32_t price_x = 104 + price_offset;
draw_text(price_x, 152 - 70, price_buffer, PRICE_FONT_SIZE);

uint8_t old_price_len = strlen(old_price_buffer);
if (old_price_len > 0)
{
  uint32_t old_price_width = old_price_len * 6 * OLD_PRICE_FONT_SIZE - OLD_PRICE_FONT_SIZE;
  uint32_t old_price_width_half = old_price_width / 2;
  uint32_t old_price_offset = 192 / 2 - old_price_width_half;
  uint32_t old_price_x = 104 + old_price_offset;
  uint32_t old_price_y = 152 - 10 - 7 * OLD_PRICE_FONT_SIZE;
  draw_text(old_price_x, old_price_y, old_price_buffer, OLD_PRICE_FONT_SIZE);
  for (uint32_t x = old_price_x - 5; x < old_price_x + old_price_width + 5; x++)
    set_pixel(x, old_price_y + 4 * OLD_PRICE_FONT_SIZE, 1);
}
```

Fig. 1. Firmware loop.

PorjectC - disp.h

```c
void set_pixel(uint32_t x,uint32_t y, uint8_t value)
{
  spi_write_byte(0x00);
  spi_write_short(x);
  spi_write_short(y);
  spi_write_byte(value);
  return;
}
void clear_screen()
{
  spi_write_byte(0x01);
}
```

Fig. 2. Display commands in `disp.h`

3) Reading Packets: The packet handling has two main parts. First part is when `read_uart()` is called. In this function (Figure 3), first the UART unit is enabled by writing `0x11` to its control register. Then a loop waits until the variable `uart_new_command` is set to 1. When that happens, it immediately sets it back to 0 and copies the data from an intermediate buffer to the buffer provided by the caller. It then finally returns the number of bytes transferred.

PorjectC - uart.h

```c
size_t read_uart(uint8_t *buffer)
{
  *(uint8_t *)UART_CTRL_REG = 0b00000011;

  while (!uart_new_command);
  uart_new_command = false;

  memcpy(buffer, recieved_bytes, recieved_bytes_num);

  return recieved_bytes_num;
}
```

Fig. 3. Reading from UART in `uart.h`

Next part is actually where `uart_new_command` is set to 1 in the first place and it is the interrupt handler for the UART unit. When the UART unit receives a new packet and its timer times out, this function runs (Figure 4). When that happens, first thing the interrupts are disabled. Then the number of available bytes is read from the UART unit's BYTECNT register. The bytes are then transferred from from the UART unit's buffer to an intermediate buffer called `recieved_bytes`. The rest of this buffer is set to 0. Finally `uart_new_command` is set to 1 and interrupts are re-enabled and program continues from where it was interrupted.

PorjectC - uart.h

```
__attribute__((interrupt("machine"))) void uart_interrupt_handler(void)
{
  // Disable interrupts
  asm volatile("csrc mstatus, %0" : : "r"(0x8));

  recieved_bytes_num = *(uint8_t *)UART_BYTECNT_REG;

  for (int i = 0; i < MAX_RECIEVED_BYTES; i++)
  {
    if (i < recieved_bytes_num)
      recieved_bytes[i] = *(uint8_t *)(UART_BUFFER_ADDR + i);
    else
      recieved_bytes[i] = 0;
  }

  // Enable interrupts
  asm volatile("csrs mstatus, %0" : : "r"(0x8));

  uart_new_command = true;

  return;
}
```

Fig. 4. Interrupt handler in `uart.h`

III. Experimental Results & Conclusion

To test the design, two test packets (Figure 5) are sent with 140ms delay between them.

```
parameter string TEST_STRING1 = "Delicious Coffee|Very good Coffee, like the best one ever|12.99$|15.99$|www.coffee.com";
parameter string TEST_STRING2= "Tea|The only tea that tastes like tea|999.99$|1499.99$|www.tea.com";
```

Fig. 5. Test packets

After around 136ms the first output is finished rendering (Figure 6). It stays on for around 4 ms before the second packet is sent. 6ms later the screen is cleared and the second tag starts generating. After around 130ms later the second tag is rendered (Figure 7) and the simulation finishes.

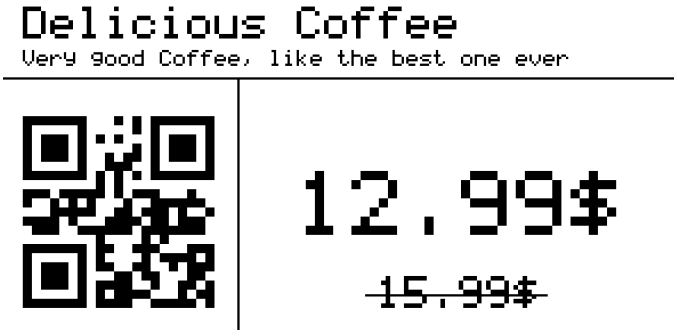


Fig. 6. First output



Fig. 7. Second output

As seen in the test, all parts of the design work perfectly and the expected results have been achieved. The QR code generation can also be confirmed by scanning it using a smartphone.

IV. Future Work

This design has some room for improvement, some of the most important ones are:

- **GPIO** : GPIO can be added to have things like status LED, battery status or a soft reset button.
- **Memory** : An SPI non-volatile memory can be added to store a series of tags and go through them one by one or even
- **Sleep Mode** : By adding a sleep mode to AFTAB or using a processor with sleep mode, the CPU can sleep while waiting for new data to arrive to save a significant amount of power.
- **Bidirectional IR Transceiver** : Upgrading the IR interface from a receiver to a full transceiver that can acknowledge responses or do battery reporting.
- **Encryption** : Encrypt the packets and decrypt them in the ESL after receiving them to stop unauthorized changes.

References

[1] ”Core-Based Embedded System Design, Project - Part C: System Implementation,” ECE 160, University of Tehran, Spring 1404-1405.

[2] S. H. Lee, J. Kim, and H. Kim, “Design of low-power electronic shelf labels for retail applications,” IEEE Transactions on Consumer Electronics

[3] “Energy-efficient e-paper display systems for portable embedded devices,” Journal of Display Technology

[4] OpenAI, “ChatGPT (GPT-5.5),” OpenAI, San Francisco, CA, USA, 2026. [Online]. Available: `https://chatgpt.com`.

[5] `https://github.com/ben-marshall/uart`

[6] `https://github.com/nandland/spi-master`

[7] `https://github.com/danielgjackson/qrtiny`

[8] M. Rajabalipanah, M. S. Roodsari, Z. Jahanpeima, G. Roascio, P. Prinetto, and Z. Navabi, “AFTAB: A RISC-V Implementation with Configurable Gateways for Security,” in *Proc. IEEE East-West Design & Test Symp. (EWDTS)*, 2021, pp. 1–6. Available: `https://github.com/RHESGroup/aftab`